

Timesheets and Overtime — Turning Punches Into Payable Hours

Prepared for: Michael Lyman, Owner, Greenway Marijuana **Legal entity:** LYMAN'S MARIJUANA L.L.C. · WA UBI 603 353 555 · EIN 46-4217016 **Slice:** books-32 · **Migration:** 0197_timesheet_workweek.sql **Screen:** /admin/books/timesheets — "Timesheets & Overtime" under Accounting **Date prepared:** 22 August 2026

The short version

Payroll is a chain, and this slice builds the first link. Before a single dollar can be calculated, withheld, deposited, or reported on a Form 941, somebody has to answer a deceptively simple question: how many hours does this person get paid for, and how many of those hours are overtime? Everything downstream — the gross wage, the federal withholding, the Social Security and Medicare, the Washington L&I hours, the Paid Family and Medical Leave premium, the W-2 at the end of the year — is a consequence of that one number. Get it wrong and every form built on top of it is wrong too, in a way that looks entirely correct.

You now have a screen that takes the punches your staff already record, sorts them into workweeks, applies the overtime rule the way the federal regulation actually states it, and shows you the arithmetic instead of hiding it. It refuses to answer when the data does not support an answer. It tells you, in plain English, exactly what is left to set up and what is blocking you. And it records who approved which pay period and when.

Along the way it caught a real defect in my own work — a daylight-saving-time bug that would have quietly stopped paying people for hours worked, starting in March 2027, after passing every test you would have run at your January 1st cutover. That story is in this report because it is the single best argument for why we are building this the slow way. I will come back to it.

Why overtime is the most dangerous arithmetic in payroll

Most payroll mistakes are recoverable. You withhold slightly too little federal income tax, the employee owes slightly more in April, nobody is harmed. You misclassify a general-ledger account, you reclassify it. Overtime is different, and it is different for three reasons that compound.

The first is that unpaid overtime accrues silently. Nobody notices. The employee sees a paycheck that looks normal, because they do not do the calculation, and you see a payroll cost that looks normal, because it is slightly lower than it should be. There is no bounce, no rejection, no notice in the mail. The error simply repeats every two weeks until somebody finally checks.

The second is that the remedy is not "pay the difference." Under both federal and Washington law, unpaid overtime carries additional exposure — back wages, and in the federal scheme liquidated damages that can double the amount, plus attorney fees. A modest per-employee shortfall across fifteen employees across two years stops being modest very quickly.

The third is the one that actually catches careful people, and it is the reason this screen is shaped the way it is. **Overtime is owed per workweek, not per pay period.** Your pay periods are two weeks long. If the software adds up the whole fortnight and pays overtime on anything past eighty hours, it will look right, feel right, produce sensible-looking numbers, and systematically underpay anyone who has a heavy week followed by a light one. This is not my opinion. The Department of Labor wrote a regulation whose entire purpose is to say so, and that regulation is quoted verbatim on your screen. From 29 CFR § 778.104:

"The Act takes a single workweek as its standard and does not permit averaging of hours over 2 or more weeks."

Read that again with a two-week pay period in mind. A person who works thirty hours one week and fifty hours the next has worked eighty hours in the period. Eighty is not more than eighty, so a period-level calculation pays zero overtime. The correct answer is ten hours of overtime, because week two stands on its own. At a twelve-dollar hourly rate that is sixty dollars of overtime premium, in one period, for one person. Multiply by twenty-six periods and a handful of employees and you have found a five-figure liability that no report would ever have shown you.

The screen therefore refuses to display a period total until it has shown you each workweek separately, on its own line, with its own overtime figure. The layout is not a style choice. It is the control.

What the screen actually does

You pick a tax year and a pay period. The system reads every time punch that falls inside that period, groups them by employee, then groups each employee's punches into workweeks using the anchor day you chose during setup. For each workweek it shows total hours, regular hours, and overtime hours. Then, and only then, it shows the period total and the gross pay.

The gross pay is broken into two components, and the reason is worth understanding because it is how the regulation itself describes the calculation. From 29 CFR § 778.110(a), quoted verbatim on the screen:

"If the employee is employed solely on the basis of a single hourly rate, the hourly rate is the 'regular rate.' For overtime hours of work the employee must be paid, in addition to the straight time hourly earnings, a sum determined by multiplying one-half the hourly rate by the number of hours worked in excess of 40 in the week. Thus a \$12 hourly rate will bring, for an employee who works 46 hours, a total weekly wage of \$588 (46 hours at \$12 plus 6 at \$6). In other words, the employee is entitled to be paid an amount equal to \$12 an hour for 40 hours and \$18 an hour for the 6 hours of overtime, or a total of \$588."

Notice the structure, because it repays a slow read. The regulation does not simply say "pay one and a half times the rate for the overtime hours." It gives the same five hundred eighty-eight dollars **two different ways**. First as forty-six hours at twelve dollars — five hundred fifty-two dollars — plus a premium of six dollars on each of the six overtime hours, which is thirty-six dollars. Then again as forty hours at twelve dollars, four hundred eighty, plus six hours at eighteen dollars, one hundred eight.

Both are correct and both total five hundred eighty-eight. Your system reports the **first** split — straight time on every hour worked, plus a separately identifiable overtime premium — and it does that deliberately, because the premium has to be separately identifiable when it reaches the W-2 and the Form 941. Your screen shows both lines and the total, so the arithmetic is auditable rather than asserted.

I have made the regulation's example executable. The test in this repository proves the engine produces exactly five hundred eighty-eight dollars, and proves it against **both** of the regulation's decompositions, so a bug that got the total right while splitting it wrongly cannot hide. Those numbers were not chosen by me. They were printed by the federal government, and the software either reproduces them or the build fails.

When the system refuses to answer

There are ten specific situations in which this engine declines to produce a number, and each one names the employee, names the problem, and tells you what to do about it. It never estimates. This is the single most important design decision in the whole slice, so let me be direct about why.

A payroll engine that guesses is worse than no payroll engine at all, because it launders uncertainty into confidence. If somebody forgot to clock out, the honest answer is "I do not know how long this shift was." A system that fills in eight hours has not solved the problem; it has hidden it, and it has attached your name to the fabrication. Some of these refusals will be annoying. That is the point — the annoyance is the signal, and it arrives before the money moves rather than after.

The engine refuses when there is no workweek anchor on file, because overtime cannot be computed without knowing where a week begins. It refuses on an open punch, where somebody clocked in and never clocked out. It refuses on a negative span, where the clock-out precedes the clock-in, which usually means a manual correction went in backwards. It refuses on overlapping punches for the same person, because a human being cannot be in two shifts at once and paying both is paying twice. It refuses on a punch that falls outside the period it was selected for. It refuses when an hourly employee has no rate on file. It refuses when somebody marked exempt from overtime is nonetheless being paid hourly, because those two facts contradict each other and one of them is wrong. It refuses when a salaried employee has punches attached, for the same reason. It refuses on impossible period dates. And it refuses on an implausible shift — one longer than a full day, which is almost always a missed clock-out somebody closed later rather than a genuine thirty-one-hour stretch of work.

Ten refusals, ten plain-English explanations, no guesses. A refused employee is excluded from the period total rather than counted as zero hours, because a total that quietly includes a zero for somebody whose punches could not be read is a total that looks complete and is not.

The part you asked for: the system knows where you are

You told me you liked that the other screens track what you have finished and what is still outstanding, and asked for the same thing here. It is there, and it is stricter than the others, because this screen has real dependencies.

There are five setup steps, and they are genuinely ordered — not cosmetically ordered, but ordered because each one physically cannot be done before the one above it. You choose the day your workweek starts. You say who is entitled to overtime. You lay out the pay periods for the year. You approve the period you are about to run. You clear the punch problems. Three of those five block computation entirely; the progress panel tells you which, and the interface will not let you begin in the middle and walk into a wall.

Each step carries its own explanation of why it exists, and each is tied to the specific regulation that makes it necessary, so the panel is teaching the reason rather than just issuing an instruction. And every fact the panel reports — how many employees still need a classification, how many are marked exempt without a written reason, how many pay periods exist against how many there should be — is read from your actual database. Nothing on that panel is a placeholder, and nothing is defaulted. If a fact is unknown, the panel says it is unknown rather than inventing a comfortable value.

The first step deserves a note. The day your workweek starts has **no default** in this system. Not Sunday, not Monday, nothing. The database column itself refuses to hold a guess. That is unusual and it is deliberate: a guessed anchor does not produce an error, it produces confident wrong overtime, because the hours get sorted into the wrong weeks and the forty-hour line lands in the wrong place. An empty field you have to fill in is a small annoyance. A wrong number you never questioned is a lawsuit.

Four things you need to know

One: the 2027 Washington minimum wage is not on file, and your first payroll is January 1st 2027

This is not a defect. It is the system correctly declining to guess, and it is the item you should put a calendar reminder against right now.

Washington's minimum wage is not a fixed number. Under RCW 49.46.020(2)(b) the Department of Labor and Industries recalculates it every year against the federal Consumer Price Index for Urban Wage Earners and Clerical Workers, and publishes the new figure by **September 30th**, effective the following **January 1st**. The 2026 figure on file is seventeen dollars and thirteen cents an hour, up from sixteen dollars sixty-six in 2025.

Your first payroll runs on January 1st 2027. That is the exact day a new minimum wage takes effect. The 2027 figure will be announced on September 30th 2026 — about five weeks from the date of this report — and until that announcement exists, this system refuses to test any 2027 wage rate against a floor rather than quietly reusing the 2026 number. Reusing it would be the obvious shortcut and it would be wrong, because the floor rises every single year without exception, so last year's number is guaranteed to be too low.

What you need to do: when L&I publishes the 2027 minimum wage at the end of September, send it to me and I will put it on file. Until then, any wage-floor check on a 2027 date will refuse. That refusal is the system working.

Two: I found a real defect in my own code, and the way it hid is the lesson

While testing the layer that reads punches out of the database, I found that the query selecting punches for a pay period was written with a hardcoded time-zone offset of minus eight hours. Minus eight is Pacific **Standard** Time. Washington observes daylight saving from the second Sunday in March to the first Sunday in November, during which the true offset is minus seven.

I measured the consequence rather than reasoning about it. For a pay period running June 5th to June 18th 2027, both ends of the punch window landed sixty minutes late. The practical effect: an employee clocking in at half past midnight on the first day of the period would **not** be included, and their hours would silently go unpaid. Simultaneously, a punch belonging to the *next* period would be dragged in, at which point the engine would refuse it as falling outside the period — so you would get a confusing refusal on one end and silent underpayment on the other, twice a year, every year.

Here is the part that matters. I ran the same comparison against a January period, and the variance was **zero**. Standard time is in effect in January. Minus eight is correct in January. Your first payroll is January 1st 2027, so this bug would have passed every test you ran at cutover, produced perfect numbers for ten weeks, and then started losing hours the second Sunday in March — by which point you would have had no reason to suspect the timesheet engine and every reason to suspect the people reporting the discrepancy.

The fix was not to write new date-handling code. The correct daylight-aware conversion already existed elsewhere in this application, and the hour-grouping half of the engine was already using it. The two halves simply disagreed. That is the ordinary shape of this kind of defect: not an exotic error, but two adjacent pieces of correct-looking code holding different assumptions. The measured variance, the January control, and the reason for the fix are all written into the source file so that nobody — including me — quietly reverts it later.

Three: I caught two of my own tests being decorative

There is a warning on this screen that fires when a pay period does not divide cleanly into whole workweeks, because that is the condition under which overtime can be under-reported across a period boundary. I had written a test to prove that warning was wired up.

As a matter of standing practice I do not trust a test until I have watched it fail. So I deliberately broke the warning — rewrote its condition so it could never display — and ran the test. **It passed.** The test was checking that the warning's name appeared in the file, and the name was still there, sitting inside a branch of code that could no longer be reached. The test was decoration. It would have reported green forever while the protection it was guarding was switched off.

The second one is subtler and I think more instructive. This screen reads three things from the database, and if any read fails it is supposed to say so in plain language. I broke it so that the failure was still *detected* but its explanation was thrown away — the screen would know something had gone wrong and show you nothing at all. **That test passed too.** A blank screen where the guidance should be is precisely the Sage behaviour you described to me at the very beginning: stopped, with no explanation, and no idea what to do next. My test had proved the failure was noticed without ever proving you would be told.

Both gates now check the condition and the visible result separately. For each one I added two further deliberate breakages to confirm the whole category was closed rather than the single instance I happened to trip over. The wiring on this screen was then broken twenty-one distinct ways on purpose; all twenty-one were caught. A twenty-second change that altered only a comment was deliberately allowed through, to prove the tests are reading the code rather than reacting to any edit at all.

Four: I corrected a piece of my own guidance that was wrong

The screen shows you the regulation's own worked example and then explains what it means in plain English. That explanation — my words, not the government's — said the system splits the five hundred eighty-eight dollars as four hundred eighty of straight time and one hundred eight of overtime.

It does not. It reports five hundred fifty-two and thirty-six.

Both pairs are legitimate readings of the regulation, as described earlier, and both add to the same total, so nothing was ever *calculated* wrongly. But the sentence explaining the system was describing a split the system does not produce, which means that if you had ever reconciled the screen against my explanation you would have found a discrepancy that did not exist and gone looking for a bug that was not there. Wrong guidance costs you time and confidence even when the arithmetic is right. The explanation now states which split we report and why the separately identifiable premium is the one the W-2 and the 941 need. The verbatim quotation itself was not touched — it never is.

What was verified before this shipped

The full test suite runs three hundred eighty-three files and eight thousand one hundred sixty-five individual tests, and every one passes. The TypeScript compiler reports no errors. The linter reports no errors and no warnings in any of the files in this slice.

Every verbatim quotation in this application — one hundred ninety-five of them now — is checked character by character against the mirrored source document, so a regulation cannot drift into a paraphrase.

The database migration was applied against a real PostgreSQL 15 server built the same way the build server builds it: all one hundred ninety-seven migrations in order, and the new one applied a second time to confirm re-running it changes nothing. I also called the migration's own audit function as a non-owner and confirmed it refuses — proving the owner-only restriction is genuinely enforced by the database rather than merely written down.

One existing test failed when I added the new screen, and it was right to. There is a gate that pins the exact list of screens in the Accounting menu, precisely so that a new item cannot appear without a person noticing. I updated the list and left the gate exactly as strict as it was.

What this feeds, and what comes next

The hours this screen produces are the input to everything. Gross wages flow into federal income tax withholding, Social Security, Medicare, and the employer match. The hours themselves — not the dollars — are what Washington L&I bills industrial insurance on, at your risk class 6403. The wages drive the state unemployment report, the Paid Family and Medical Leave premium, and the WA Cares contribution. At year end the same figures become the W-2s and the W-3 that covers them.

What is not built yet, so that you know exactly where the edges are: year-to-date figures are not yet persisted between pay runs, which the W-2 and the Form 941 will both need; the tax total on a pay run is still stored as a single combined number rather

than split into the individual taxes those forms itemise; and the pay run screen still asks you to type the net pay rather than computing it. Each of those is a named, scheduled piece of work, not an oversight.

The boundary of this whole project is unchanged and worth restating: we prepare the data. We are not a filing agent. Every figure this system produces is designed to be checked by you and then filed by you, through the proper channel, with the arithmetic visible the whole way so that you can defend it.

What I need from you

The 2027 Washington minimum wage, once L&I announces it on September 30th 2026.

Beyond that, when you next have time at the keyboard, walk into `/admin/books/timesheets` and look at the progress panel. It will tell you which of the five setup steps are outstanding for your business. The first one — the day your workweek starts — is the one with no default and no safe guess, and it is the one everything else waits on.